INDEX

	contract name	
	Freelancer Escrow Smart Contract	**2-5**
	FreelancerSoulBoundToken	**6**
	MY token	**7-8**
	NDASoulBoundToken	**9-10**
	StakingRewards	**11-12**

Freelancer Escrow Smart Contract

Audit Scope: Escrow lifecycle, voting mechanism, oracle governance, and fund flow integrity.

Prepared in professional format inspired by industry-standard security firms.

1. Executive Summary

This report evaluates the security posture and architectural integrity of the Freelancer Escrow smart contract system. The protocol implements token-based escrow with third-party oracle voting for dispute resolution between businesses and freelancers. The most critical issue identified is improper enforcement of escrow lifecycle immutability. Escrow identifiers can be reinitialized, allowing state mutation and potential manipulation of majority thresholds. While arithmetic logic and reentrancy protections are largely correct, lifecycle boundaries are insufficiently enforced.

2. Risk Summary

ID	Title	Severity
H-01	Escrow Reinitialization & State Overwrite	High
M-01	Voting Eligibility Mismatch	Medium
M-02	Duplicate Oracle Addresses Allowed	Medium
M-03	Vote State Persistence	Medium
M-04	Unsafe State Update Order in depositFunds()	Medium

3. Findings

H-01: Escrow Reinitialization and State Overwrite

```
function createEscrow(
    string calldata _escrowId,
    address[] calldata _votingOracles,
    address _freelancer,
    address _tokenAddress
) external {
    require(bytes(_escrowId).length > 0, "EmptyEscrowId");
    require(_votingOracles.length % 2 != 0, "InvalidOracleCount");
    for (uint256 i = 0; i < _votingOracles.length; i++) {
        require(s_oracles[_votingOracles[i]], "NotAValidOracle");
    }
    Escrow storage newEscrow = s_escrows[_escrowId];
```

Description: The createEscrow() function does not enforce uniqueness of escrow identifiers. Any caller may recreate an existing escrow ID and overwrite its configuration, including oracle set, freelancer address, business address, and token address. Impact: Since vote counters are stored separately and majority is calculated dynamically from the oracle array length, an attacker may reduce quorum requirements and prematurely trigger release or refund logic. Root Cause: Escrow identity is treated as a reusable mapping key rather than an immutable lifecycle instance. Recommendation: Enforce escrow ID uniqueness prior to initialization and prevent state mutation after creation.

M-01: Voting Eligibility Mismatch

```
function vote(
    string calldata _escrowId,
    bool _release
) external onlyOracle {
    require(!s_hasVoted[_escrowId][msg.sender], "AlreadyVoted");
    s_hasVoted[_escrowId][msg.sender] = true;
```

```
modifier onlyOracle() {
    require(s_oracles[msg.sender], "OnlyOracle");
    _;
}
```

The vote() function validates voters against a global oracle registry but does not enforce membership within the escrow-specific oracle list. This weakens dispute isolation.

M-02: Duplicate Oracle Addresses

createEscrow() does not enforce uniqueness of oracle addresses. Duplicate entries distort quorum calculation.

```
    function createEscrow(
        string calldata _escrowId,
        address[] calldata _votingOracles,
        address _freelancer,
        address _tokenAddress
    ) external {
        require(bytes(_escrowId).length > 0, "EmptyEscrowId");
        require(_votingOracles.length % 2 != 0, "InvalidOracleCount");
        for (uint256 i = 0; i < _votingOracles.length; i++) {
            require(s_oracles[_votingOracles[i]], "NotAValidOracle");
        }
```

M-03: Vote State Persistence

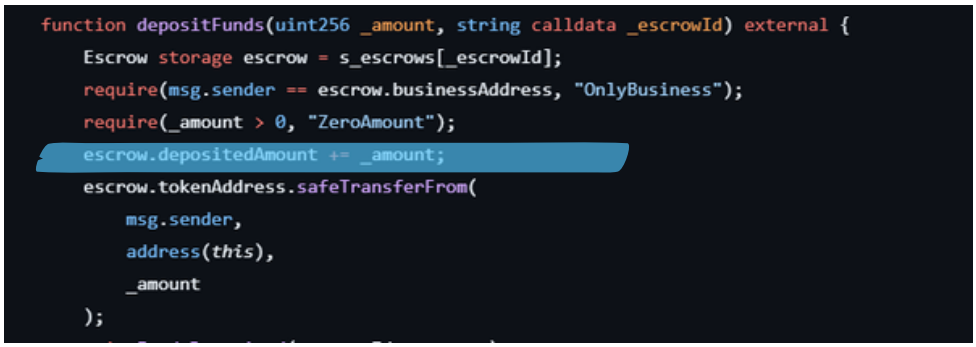
```
mapping(string => mapping(address => bool)) private s_hasVoted;
mapping(string => uint256) private s_releaseVotes;
mapping(string => uint256) private s_refundVotes;
```

```
function createEscrow(
    string calldata _escrowId,
    address[] calldata _votingOracles,
    address _freelancer,
    address _tokenAddress
) external {
    require(bytes(_escrowId).length > 0, "EmptyEscrowId");
    require(_votingOracles.length % 2 != 0, "InvalidOracleCount");
    for (uint256 i = 0; i < _votingOracles.length; i++) {
        require(s_oracles[_votingOracles[i]], "NotAValidOracle");
    }
    Escrow storage newEscrow = s_escrows[_escrowId];
    newEscrow.escrowId = _escrowId;
    newEscrow.votingOracles = _votingOracles;
    newEscrow.freelancerAddress = _freelancer;
    newEscrow.businessAddress = msg.sender;
    newEscrow.depositedAmount = 0;
```

So the escrow struct can be overwritten.
But the vote mappings are NOT cleared.

Vote counters persist independently of escrow struct reinitialization, enabling inconsistent state behavior if escrow parameters are mutated.

M-04: Unsafe State Update Order



```
function depositFunds(uint256 _amount, string calldata _escrowId) external {
    Escrow storage escrow = s_escrows[_escrowId];
    require(msg.sender == escrow.businessAddress, "OnlyBusiness");
    require(_amount > 0, "ZeroAmount");
    escrow.depositedAmount += _amount;
    escrow.tokenAddress.safeTransferFrom(
        msg.sender,
        address(this),
        _amount
    );
```

The depositFunds() function updates the escrow's depositedAmount before executing the external safeTransferFrom call. While standard ERC20 tokens do not introduce reentrancy risk, updating state prior to external interaction deviates from the recommended checks-effects-interactions pattern. If a malicious or non-standard token were used, this ordering could introduce unexpected behavior. Reordering the operations or adding nonReentrant protection would improve defensive robustness.

4. Governance & Architectural Observations

Oracle management is centralized under contract ownership. Staking validation is only checked at oracle addition and not enforced continuously. The escrow lifecycle lacks explicit finalization state flags. Multiple mapping-based identifiers allow silent overwrites, indicating inconsistent lifecycle enforcement.

5. Conclusion

The contract demonstrates sound arithmetic majority voting logic and correct reentrancy protection for fund release. However, escrow immutability is not enforced, introducing systemic integrity risk. The highest priority remediation is enforcing escrow ID uniqueness to prevent state overwrite and quorum manipulation.

Contract: FreelancerSoulBoundToken

Solidity Version: ^0.8.28
Inheritance: ERC721, AccessControl, ReentrancyGuard

1. Executive Summary

The FreelancerSoulBoundToken contract implements a role-controlled SoulBound ERC721 token system representing freelancer identity, reputation, skills, and achievements. After comprehensive review of access control, storage handling, state transitions, achievement logic, skill endorsement mechanics, soulbound enforcement, and denial-of-service vectors, no Critical or High-severity vulnerabilities were identified. The contract is structurally sound but centralized by governance design.

2. Findings Summary

- No reentrancy vulnerabilities detected.
- No arithmetic overflow/underflow issues (Solidity 0.8+ safe).
- Access control correctly enforced via role-based restrictions.
- Soulbound transfer mechanism correctly prevents token transfers.
- Minor logic issue: Achievement duplication possible (no uniqueness check).
- Potential scalability concern: Unbounded loop over skillVerified array may increase gas costs.
- System is centralized; admin holds significant authority over minting, updating, and burning.

3. Key Invariants Verified

- One address can hold at most one token.
- Burning a token correctly clears associated mappings and profile data.
- Skill arrays remain synchronized (endorsedSkills.length == skillVerified.length).
- averageRating is bounded (<= 100).
- Token ownership changes only through minting and burning.
- Role-based access control restricts state mutation.

4. Conclusion

The contract demonstrates solid engineering practices and proper access control mechanisms. No critical security vulnerabilities were identified. Minor improvements may be considered for achievement uniqueness enforcement and scalability optimization. Overall Risk Level: LOW (Technical) / MODERATE (Governance Centralization)

Project: MyToken (DEHIX - DXUT)

Compiler Version: Solidity 0.8.28 A

---

1. Executive Summary

The MyToken contract is a minimal ERC20 implementation extending OpenZeppelin's ERC20 and AccessControl modules. The contract enables role-based minting and burning functionality.

After comprehensive manual review of the provided source code, no critical, high, or medium severity vulnerabilities were identified. The contract has a minimal attack surface and leverages battle-tested OpenZeppelin components.

2. Scope

- Single contract: MyToken.sol
- Dependencies: OpenZeppelin ERC20 and AccessControl
- No upgradeability pattern detected

3. Architecture Overview

- Inherits ERC20 and AccessControl
- Defines MINTER_ROLE and BURNER_ROLE
- Deployer receives DEFAULT_ADMIN_ROLE, MINTER_ROLE, and BURNER_ROLE
- Minting restricted to MINTER_ROLE
- Burning restricted to BURNER_ROLE

4. Security Review

- Proper use of role-based access control (onlyRole modifier).
- No public mint or unrestricted token issuance.
- No reentrancy attack surface (no external calls).
- No arithmetic vulnerabilities (Solidity 0.8.x built-in checks).
- No state corruption risks identified.

5. Informational Notes



```
    function burn(address from, uint256 amount) public onlyRole(BURNER_ROLE) {
        _burn(from, amount);
    }
}
```

The BURNER_ROLE allows authorized addresses to burn tokens from any account without requiring approval. This behavior appears intentional and consistent with role-based administrative control.

6. Conclusion

The MyToken contract is structurally sound and follows secure development practices. No exploitable vulnerabilities were discovered within the defined scope of this review.

Overall Risk Assessment: LOW

Contract: NDASoulBoundToken

Language: Solidity ^0.8.28

Dependencies: OpenZeppelin ERC721, AccessControl, ReentrancyGuard

---

1. Executive Summary

The NDASoulBoundToken contract implements a non-transferable ERC721 token to represent NDA lifecycle agreements between a business owner and a freelancer. The contract enforces structured state transitions, controlled token burning, and role-based access control.

After comprehensive review, no critical or high-severity vulnerabilities were identified. The contract demonstrates controlled lifecycle enforcement and consistent burn logic. Minor design and gas-efficiency observations are documented below.

2. Scope of Review

- State variables and storage mappings
- Access control and role enforcement
- Lifecycle state transitions
- Token migration and burn mechanisms
- Expiration and completion logic
- Violation reporting
- Soulbound transfer prevention
- View functions and event emissions

3. Security Assessment

- No reentrancy vulnerabilities identified (no external calls or value transfers).
- No ID reuse or token collision risks (monotonic counter increment).

- Burn logic maintains consistency between ERC721 state and internal struct.
- Soulbound enforcement correctly overrides transfer functions.
- State transitions are properly gated and validated.

4. Observations & Recommendations

- Unbounded string inputs (content, signatures, violation reason) may lead to increased gas costs and storage bloat. Consider storing content hashes or IPFS references instead of full strings.
- Expiration is enforced reactively via a callable function. Consider optionally enforcing expiration checks during signing if business logic requires strict timing.
- Lifecycle enum includes 'Active' state but it is not explicitly assigned in the current implementation. Review whether this state is required.
- Historical token IDs remain stored in mapping arrays after burn. This is not a vulnerability but may affect indexing clarity.

5. Conclusion

Based on the review performed, the NDASoulBoundToken contract is structurally sound and does not contain critical or high-severity security vulnerabilities. The implementation demonstrates appropriate lifecycle controls and state integrity. Minor design improvements may enhance gas efficiency and clarity, but no immediate security risks were identified.

Contract: StakingRewards

Compiler Version: Solidity ^0.8.28

Auditor: Mahin Modasiya **Date:**

2026

---

1. Executive Summary

The StakingRewards contract implements a time-based ERC20 staking mechanism following a Synthetix-style reward distribution model. Users stake a token and earn rewards proportionally over a fixed duration. The contract includes reward merging logic, reentrancy protection, and emission cap validation.

No critical or high-severity vulnerabilities exploitable by external users were identified. However, several medium and low-severity issues and design considerations were observed.

2. Findings

M-01: Missing Zero Duration Validation

```
function setRewardsDuration(uint256 _duration) external onlyOwner {
    require(finishAt < block.timestamp, "reward duration not finished");
    duration = _duration;
    emit RewardsDurationSet(_duration);
}
```

```
function notifyRewardAmount(
    uint256 _amount
) external onlyOwner updateReward(address(0)) {
    if (block.timestamp >= finishAt) {
        rewardRate = _amount / duration;
    } else {
        uint256 remainingRewards = (finishAt - block.timestamp) *
            rewardRate;
        rewardRate = (_amount + remainingRewards) / duration;
    }
```

The function setRewardsDuration(uint256 _duration) does not validate that _duration is greater than zero. If duration is set to zero, subsequent calls to notifyRewardAmount() will revert due to division by zero.

Impact: Reward distribution can become permanently unusable (Denial of Service).

Recommendation: Add require(_duration > 0, "duration = 0");

L-01: Fee-on-Transfer Token Assumption



```
function stake(
    uint256 _amount
) external nonReentrant updateReward(msg.sender) {
    require(_amount > 0, "amount = 0");
    // Effects
    balanceOf[msg.sender] += _amount;
    totalSupply += _amount;
    // Interaction
    stakingToken.safeTransferFrom(msg.sender, address(this), _amount);
    emit Staked(msg.sender, _amount);
}
```

The contract assumes stakingToken and rewardsToken are standard ERC20 tokens without transfer fees. Fee-on-transfer tokens may cause accounting mismatches or distribution inconsistencies.

Recommendation: Document token assumptions or compute actual received amounts using balance delta tracking.

The contract increases balance by _amount

but never checks how many tokens were actually received.

If the token charges a fee:
User sends 100
Contract receives 95
But balanceOf is credited 100
Accounting mismatch.

3. Governance & Centralization Observations

The contract grants the owner unilateral authority to:

- Set reward duration
- Start new reward emission cycles
- Penalize and confiscate user stakes
- Reset user rewards

These powers are not vulnerabilities but represent a trust-based governance model. Users must trust the owner not to misuse administrative privileges.

4. Final Assessment

The reward accounting logic, emission merging mechanism, and reentrancy protections are implemented correctly. The contract does not exhibit external fund-drain vulnerabilities. Risks identified are primarily related to governance authority and token behavior assumptions.